

Contents

Guide 2 — The Recommendation Engine (TOPSIS), the simple version

1. The idea, in three sentences

2. How it works — five steps

3. The one formula to remember

4. Where each part comes from (the research)

5. A worked example

6. The files

7. Jury questions — ready answers

Appendix — the full calculation (how C = 0.60)

1

1

1

1

2

3

3

3

Guide 2 — The Recommendation Engine (TOPSIS), the simple version

How Mobile Match Algeria picks the best 3 plans for a user — in plain words, with the research each part comes from, and ready answers for the jury.

1. The idea, in three sentences

The user gives their **budget**, their **data need**, and **ranks** what matters most (price, data). The engine throws away every plan that doesn't fit the hard limits, then ranks the rest by **how close each plan is to the “ideal” plan**. It uses no AI and no other users' data, only this user's profile, so it works from day one.

2. How it works — five steps

1. **Filter**: drop any plan over budget, or with the wrong operator / type / network.

2. **Weights**: turn the user's ranking into numbers — price #1, data #2 → **0.75 / 0.25**.

3. **Compare to two reference plans**: the **ideal** (cheapest *and* most data) and the **worst** (priciest, least data).

4. **Closeness**: measure how near each plan is to the ideal versus the worst — a score from **0 (worst) to 1 (best)**.

5. **Rank by closeness**, show the **top 3** (the score becomes the match %).

3. The one formula to remember

closeness C = (distance to the worst plan)

(distance to the ideal) + (distance to the worst)

The “distance” is a plain **Euclidean (straight-line) distance** across the criteria. A plan sitting right on the ideal gets C = 1; one sitting on the worst gets C = 0.

4. Where each part comes from (the research)

Nothing here is invented — each step is a published method:

Part of the engine	What it does	Comes from (research)
Hard filters	remove plans that don't fit	Conjunctive (constraint) method — a non-compensatory MADM screening step (Hwang & Yoon, 1981)
Weights from a ranking	“price #1, data #2” → 0.75 / 0.25	Rank-Order Centroid — Barron & Barrett (1996)
Ranking by closeness to the ideal	the whole formula in §3	TOPSIS — Hwang & Yoon (1981); recent guide: Taherdoost & Madanchian (2023)

Every step — filtering, weighting, and **the ranking itself** — is a published method. **What is ours are configuration choices, not formulas:** - *which* attributes are hard filters (budget, operator, type, network) versus ranked criteria (price, data), and that the **budget is a hard ceiling**; - two small **data-encoding conventions**: treating an “unlimited” allowance as the best value on its criterion (so it beats any finite amount), and falling back to **equal weights** when the user gives no ranking.

None of these computes a score; the scoring is entirely the cited methods above.

5. A worked example

The user sets **budget 2000 DA**, **need 10 GB**, and ranks **price #1, data #2** (so the ROC weights are **0.75 for price, 0.25 for data**). Three plans pass the budget filter:

Plan	Price	Data
A	1000 DA	8 GB
B	1100 DA	15 GB
C	600 DA	4 GB

Step 1 — build the two reference plans (per criterion, across these three):

- **Ideal** = best of each column → cheapest **600 DA** + most data **15 GB**.
- **Worst** = worst of each column → priciest **1100 DA** + least data **4 GB**.

Step 2 — for each plan, measure its distance to the ideal and to the worst, then $C = (\text{distance to worst}) / (\text{distance to ideal} + \text{distance to worst})$. Price counts about 3× as much as data here, because of the 0.75 weight:

Plan	How it sits	Closeness C	Rank
C (600 DA / 4 GB)	is the cheapest → closest to the ideal	0.60	1st
B (1100 DA / 15 GB)	priciest, but the most data	0.40	2nd
A (1000 DA / 8 GB)	mid price, mid data	0.26	3rd

Result: C → B → A. C wins because price is weighted most and C is the cheapest. B beats A because, at a similar price, B has far more data. If the user ranked **data first**, the 0.75 weight would shift to data and **B** would jump to the top — that's how the ranking follows the user's priority.

6. The files

- [recommendation.ts](#) — the engine: filters, rocWeights(), topsis().
 - [api/recommendations/route.ts](#) — the API (validated, rate-limited).
 - [recommend/page.tsx](#) — the form + ranked cards (with savings).
-

7. Jury questions — ready answers

- **What are you calculating? A Euclidean distance?** → Yes. Each plan's straight-line distance to the ideal and the worst plan, combined into a closeness score (TOPSIS).
 - **Did you invent the formula?** → No. The ranking is **TOPSIS** (Hwang & Yoon, 1981), the weights are **Rank-Order Centroid** (Barron & Barrett, 1996) — both published. Only the choice of criteria and the budget ceiling are ours.
 - **Does it work with no users?** → Yes, it's content-based (uses plan attributes + the user's profile), so it has no cold-start problem.
 - **Why TOPSIS, not AI / a weighted sum / lexicographic / AHP?** → AI needs a usage history we don't have; a weighted sum gives an unbounded, scale-sensitive score; strict lexicographic is too rigid; AHP needs many pairwise comparisons. TOPSIS is bounded, content-based, and pairs naturally with the ranking-based weights.
 - **What if no plan fits?** → The list is empty (the budget ceiling is strict); the user can raise the budget or relax a filter.
-

Appendix — the full calculation (how C = 0.60)

For the example in §5 (budget 2000; weights price 0.75, data 0.25; A 1000 DA/8 GB, B 1100 DA/15 GB, C 600 DA/4 GB). The five formulas:

1. normalise: $r = \text{value} / \sqrt{\text{sum of squares in the column}}$
2. weight: $v = w * r$
3. ideal A+ = best per column (price: smallest ; data: largest)
worst A- = worst per column (price: largest ; data: smallest)
4. distances: $S+ = \sqrt{(v_{\text{price}} - A+_{\text{price}})^2 + (v_{\text{data}} - A+_{\text{data}})^2}$ (to ideal)
 $S- = \sqrt{(v_{\text{price}} - A-_{\text{price}})^2 + (v_{\text{data}} - A-_{\text{data}})^2}$ (to worst)
5. closeness: $C = S- / (S+ + S-)$

Step 1 — column lengths:

- price: $\sqrt{1000^2 + 1100^2 + 600^2} = \sqrt{2,570,000} \approx 1603$
- data: $\sqrt{8^2 + 15^2 + 4^2} = \sqrt{305} \approx 17.46$

Steps 1–2 — normalise then × weight → the v values:

Plan	$v_price = 0.75 \times price/1603$	$v_data = 0.25 \times data/17.46$
A	0.468	0.114
B	0.515	0.215
C	0.281	0.057

Step 3 — the two reference points:

- Ideal A+ = (price **0.281** [C, cheapest], data **0.215** [B, most data])
- Worst A- = (price **0.515** [B, priciest], data **0.057** [C, least data])

Steps 4–5 — for C (its $v = 0.281$ price, 0.057 data):

$$S+ = \sqrt{(0.281-0.281)^2 + (0.057-0.215)^2} = \sqrt{0 + 0.158^2} = 0.158$$

$$S- = \sqrt{(0.281-0.515)^2 + (0.057-0.057)^2} = \sqrt{0.234^2 + 0} = 0.234$$

$$C = S- / (S+ + S-) = 0.234 / (0.158 + 0.234) = 0.234 / 0.392 = 0.60$$

Same for the others: **B** → $S+ = 0.234$, $S- = 0.158$ → **0.40**; **A** → $S+ \approx 0.212$, $S- \approx 0.074$ → **0.26**.

Why C wins: C is the cheapest, so on price its value sits *exactly on the ideal* (distance 0), and price carries 0.75 of the weight — so C ends up closest to the ideal and farthest from the worst, giving the highest closeness, **0.60**.

Next guide: the search engine (token parser + FTS5). Tell me when you want it.